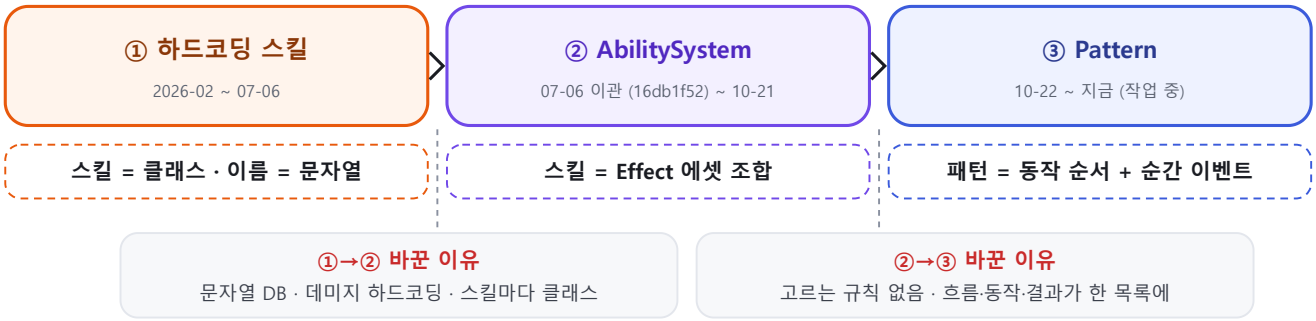
BOSS · STRUCTURE EVOLUTION

보스 전투 구조, 세 번의 모양

하드코딩 스킬 → AbilitySystem → Pattern. 같은 보스(예로니무스)의 같은 스킬들을 세 구조로 만들어 봤습니다. 단계마다 **무엇을 풀었고 무엇을 남겼는지**를 같은 기준으로 비교합니다.

흐름 — 세 단계와 바꾼 이유



세 질문이 단계마다 어디에 있었나 — 진하게 칠한 칸이 그 단계에서 바뀐 곳

	① 하드코딩	② AbilitySystem	③ Pattern
언제 어떤 패턴을 언제	**BT RandomNode 균등 추첨** 플래그 2개 + 타임라인 종료 콜백	그대로 (안 바뀜) 균등 추첨 · 플래그 2개	**BT 우선순위 + 패턴 표** 가중치 · 쿨 · 직전 제외 · HP 줄
무엇을 무엇을 실행하나	**시그널 문자열 → 스킬 클래스** BossSkillBase 파생 6종	**시그널 → AbilityData** Effect 에셋을 중첩해 조합	**Pattern = Action 순서** 순간은 시그널 → PatternEvent
어떻게 피해를 어떻게 넣나	**씬 프리팹에 하드코딩** 번개 10 · 장판 500 고정	**Effect가 보스 Stat을 읽음** AreaStrike · DealDamage	아직 ②의 Effect 사용 → 데미지 파이프라인 예정

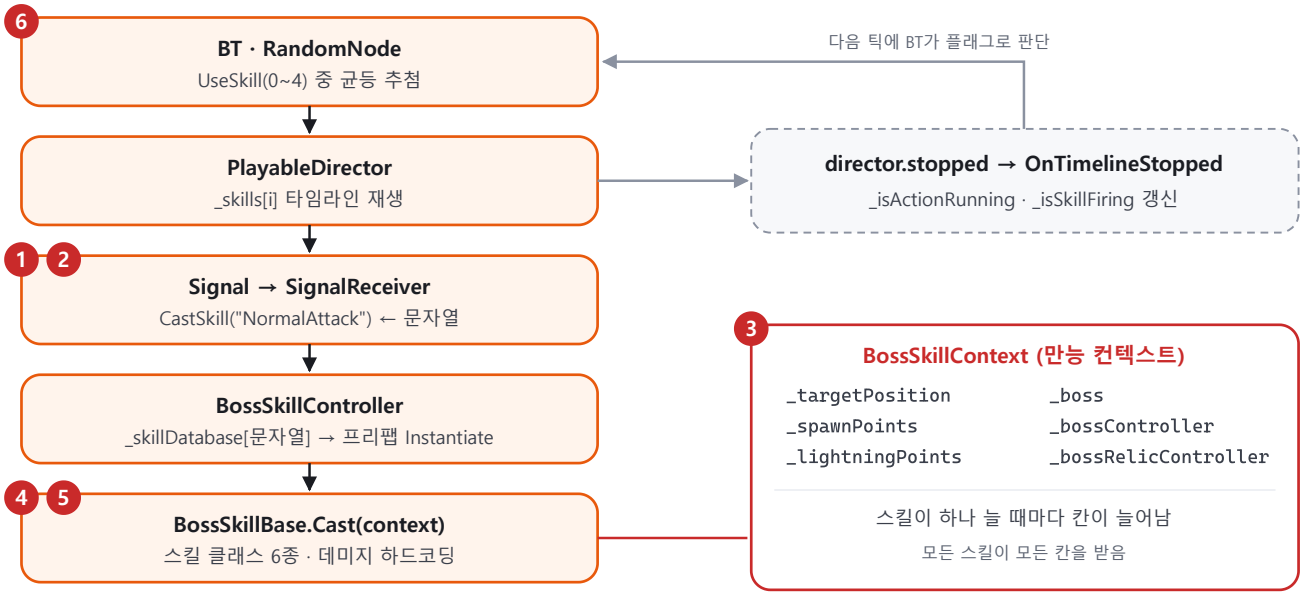
이 표를 읽는 법

②는 ①의 **'무엇을·어떻게'**를 데이터로 옮겼지만 **'언제'**는 건드리지 않았습니다. ③은 ②가 남긴 '언제'를 표로 풀고, 한 목록에 섞여 있던 흐름·동작을 층으로 나눴습니다. '어떻게'(데미지 파이프라인)는 다음 차례입니다.

① 하드코딩 스킬 — 2026-02 ~ 07-06

BT가 타임라인을 고르고, 타임라인 시그널이 **문자열**로 스킬 프리팹을 찾아 실행했습니다. 스킬 하나가 클래스 하나였고, 데미지는 씬 프리팹에 숫자로 박혀 있었습니다.

구조 — 빨간 번호는 아래 문제 목록의 번호



①이 가진 문제 6가지

1. **이중 결정** — "무슨 스킬인가"를 BT 번호(어느 타임라인)와 시그널 문자열(어느 프리팹)이 따로 정함
2. **문자열 DB** — 이름 오타는 실행 중에야 에러, 코드에서 정의로 이동할 수 없음
3. **만능 컨텍스트** — 스킬이 늘 때마다 `BossSkillContext`에 칸 추가 (6칸)
4. **데미지 하드코딩** — 씬 프리팹 숫자(번개 10 · 장판 500), 보스 Stat·크리와 단절, 공격자가 이펙트 오브젝트
5. **스킬 = 클래스 1개** — 조합 불가, 번개를 쓰는 스킬도 매번 새 클래스
6. **고르는 규칙 없음** — 5개 균등 추첨, 같은 패턴 연속, HP 기믹 불가, 플래그 2개로 진행 상태 관리

그래도 옳았던 것

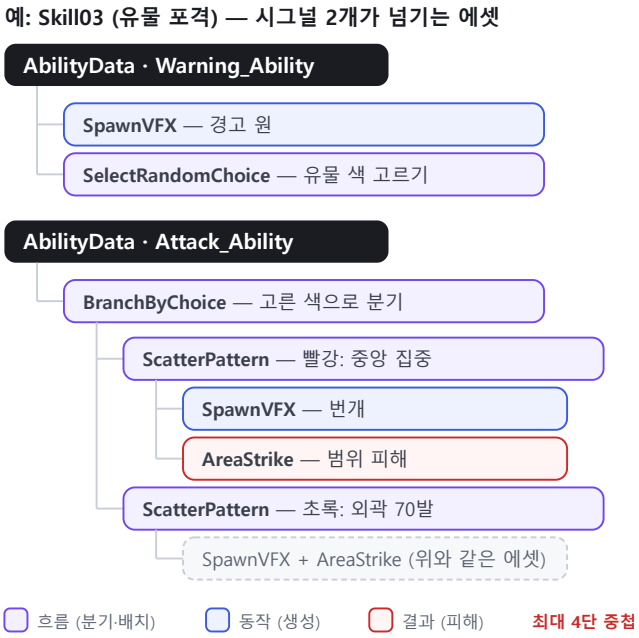
연출(타임라인)과 기계(스킬 코드)를 나눈 것. ②는 이 분리를 그대로 두고 **기계를 담는 그릇만** 바꿨습니다.

② AbilitySystem — 07-06 이관 ~ 10-21

시그널이 문자열 대신 **AbilityData 에셋**을 직접 넘기고, 스킬은 Effect 에셋의 조합이 됐습니다. 플레이어·일반 몹과 같은 `TryCast` 경로를 탑니다. 다만 BT 쪽은 그대로였습니다.

구조 — 회색은 ①에서 바뀌지 않은 부분





①의 문제	②에서	어떻게
1 이중 결정	풀림	BT는 타임라인만, 무슨 스킬인지는 시그널의 에셋 참조가 정함
2 문자열 DB	풀림	`BossSkillController` 제거, 에셋을 드래그로 연결
3 만능 컨텍스트	풀림	`AbilityContext` 는 누가·어디·대상만, 씬 참조는 인터페이스로 조회
4 데미지 하드코딩	풀림	Effect가 `ctx.CasterStat` 을 읽음, 공격자 = 보스
5 스킬 = 클래스	풀림	payload 재사용 — 평타·Skill2는 새 코드 0줄 (번개 + AreaStrike)
6 고르는 규칙 없음	그대로	BT를 건드리지 않음 — 균등 추첨 · 연속 중복 · HP 기믹 불가

②가 남긴 문제 4가지

1. **보스에겐 빈 게이트** — "쓸 수 있나" 판정이 핵심인데 보스는 6개 모두 0. 패턴 쿨은 둘 곳이 없음
2. **흐름·동작·결과가 한 목록** — 분기·생성·피해가 한 목록에 섞여 인스펙터에서 SO 4단 중첩
3. **시작·끝 신호가 클립 이벤트** — 클립 7개에 문자열로 박힌 이벤트 + `director.stopped` 콜백
4. **사망 취소가 늦음** — `_monsterCts` 가 디스폰 대기 겸용 → 쓰러지는 5초 동안 소환 진행

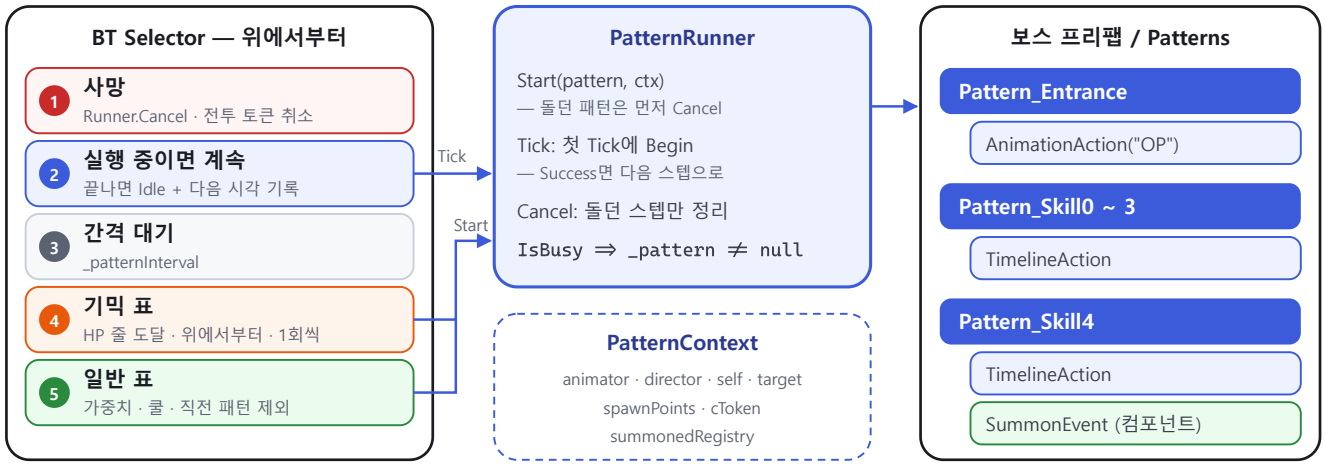
②의 진짜 성과는 남는다

플레이어·몹·보스가 `TryCast` 하나로 모인 것은 그대로입니다. ③은 **보스 쪽만** Pattern으로 옮겼고, 플레이어의 AbilitySystem(게이팅 단일화·쿨 절대시각)은 건드리지 않습니다.

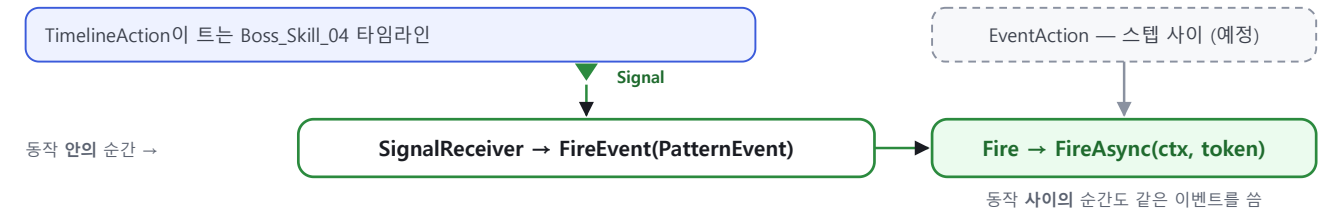
③ Pattern — 10-22 ~ 지금

BT는 **무엇을 시작할지**만 정하고, 시작한 패턴의 수명은 **PatternRunner**가 끝까지 책임집니다. 패턴은 시간이 걸리는 **Action**의 순서이고, 그 사이의 한순간에 터지는 것은 **Event**입니다.

구조



순간 이벤트가 터지는 두 길



클래스	맡는 일	지금 있는 것
Pattern	고르는 공격 단위 = Action 목록 + 사거리	입장 1 · 일반 5
PatternAction	시간 걸리는 동작 — `Begin/Tick/Cancel`	Animation · TimelineAction
PatternEvent	한순간 실행하고 끝 — `Fire` → `FireAsync`	SummonEvent
PatternRunner	패턴 하나의 수명 — 순서 진행·취소	보스가 하나 소유
Normal·GimmickPatternSlot	표의 한 칸 — 가중치·쿨 / HP 줄	보스 인스펙터의 두 표

Action — 기다려야 하면
끝날 때까지 다음으로 안 넘어감
예: 애니 · 타임라인 · 대기 · 이동

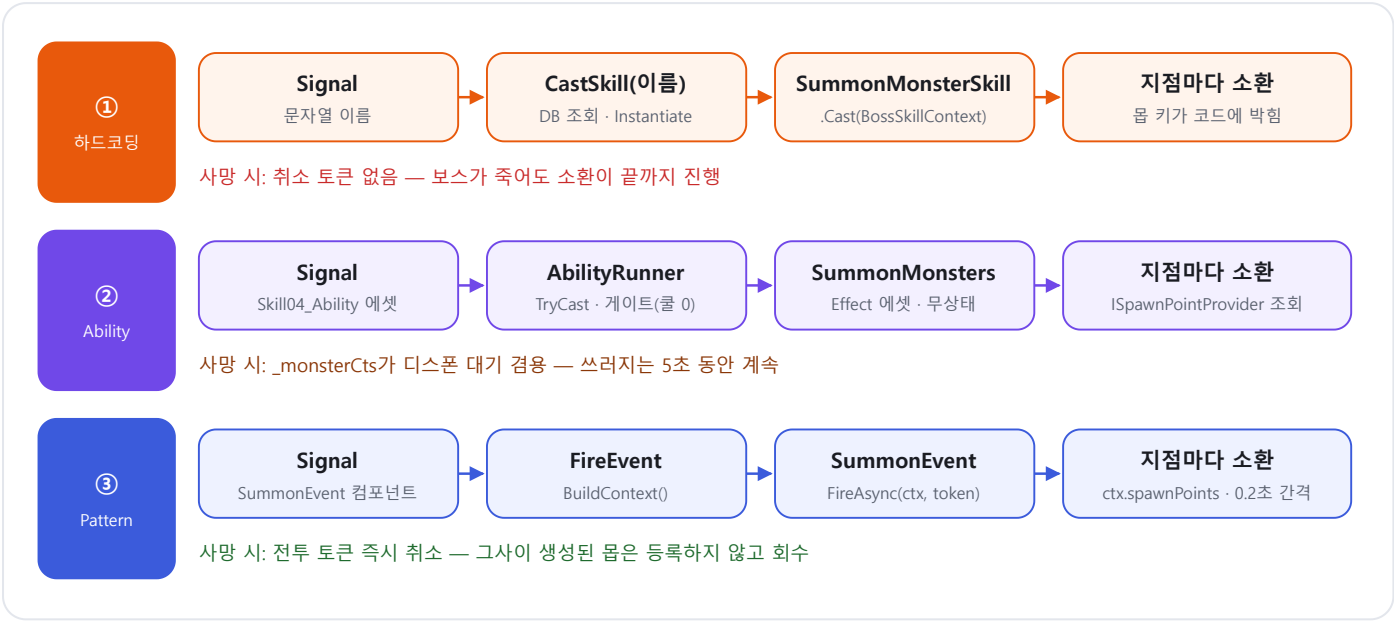
Event — 쏘고 잊으면
부르는 쪽은 기다리지 않음 (비동기는 내부 사정)
예: 소환 · 이펙트 · 사운드 · 범위 타격

③이 푼 것

1. **고르는 규칙** (①-6) → 일반 표(가중치·쿨·직전 제외) + 기믹 표(HP 줄) + BT 우선순위
2. **빈 게이트** (②-1) → 쿨은 표의 칸으로. 보스는 TryCast 게이트를 거치지 않음
3. **섞인 목록** (②-2) → 흐름 = Pattern·Runner(코드), 동작 = Action, 순간 = Event — 종류마다 클래스 하나
4. **클립 이벤트** (②-3) → 끝은 Tick이 직접 확인(Animator 상태 · `director.state`), 클립 이벤트 삭제
5. **늦은 취소** (②-4) → 전투 토큰 분리 — 사망 즉시 취소, 디스폰 타이머는 그대로

비교 같은 스킬, 세 번 — Skill4 몬스터 소환

세 단계 모두 "타임라인의 특정 순간에 맵 지점마다 몹을 소환하고 보스에 등록한다"는 같은 일을 합니다. 달라진 것은 **시그널이 무엇을 넘기는지**와 **사망했을 때 어떻게 되는지**입니다.



항목	① 하드코딩	② AbilitySystem	③ Pattern
시그널이 넘기는 것	문자열	AbilityData 에셋	SummonEvent 컴포넌트
소환 지점	BossSkillContext의 칸	캐스터에서 인터페이스로 조회	`ctx.spawnPoints` (provider에서 매번)
몹 종류	코드 안 문자열	에셋 값	컴포넌트 값
보스 등록	`_bossController` 직접 호출	`ISummonRegistry`	`ctx.summonedRegistry`
사망 시	**끝까지 진행**	**5초간 진행**	**즉시 멈춤 + 늦게 온 몹 회수**
다른 캐스터가 쓰기	**불가**	**가능 (몹·플레이어)**	**보스 프리팹 안에서**

종합 장단점 한눈에

기준	① 하드코딩	② AbilitySystem	③ Pattern
패턴을 고르는 규칙	○ 균등 추첨	○ 그대로	● 가중치·쿨·HP 줄
실행 수명 (시작·끝·취소)	○ 플래그 + 콜백	◐ Effect 실행만 관리	● Runner 한 곳
순간 타이밍 연결	◐ 시그널 + 문자열	● 에셋 참조	● 컴포넌트 참조
피해와 Stat 연결	○ 하드코딩	● Stat 주입	◐ 아직 ② 경유
재사용 범위	○ 스킬마다 클래스	● 플레이어·몹·보스	◐ 보스 프리팹 안
흐름이 한눈에 보이나	◐ 클래스마다 흩어짐	○ SO 4단 중첩	● 하이어라키 = 순서
새 동작 추가	클래스 + 문자열 등록	에셋 조합 (코드 0줄도 가능)	컴포넌트 클래스 1 + 연결
씬 참조	만능 컨텍스트	인터페이스 조회	프리팹 직접 참조 + ctx

● 충족 · ◐ 부분 · ○ 못 함

① 하드코딩

+ 연출(타임라인)과 기계(코드) 분리
+ 구조가 단순해 빨리 만듦

- 문자열 · 숫자 하드코딩
- 만능 컨텍스트
- 고르는 규칙 · 취소 없음

② AbilitySystem

+ 문자열 제거 · Stat 연결
+ payload 재사용 (코드 0줄 스킬)
+ 플레이어·몹·보스 한 경로

- 보스에겐 빈 게이트
- 흐름·동작·결과 섞인 중첩
- '언제'는 그대로

③ Pattern

+ 표로 고르는 규칙
+ Runner가 수명 단일 책임
+ Action / Event 구분 · 전투 토큰

- 패턴을 보스끼리 공유 못 함
- 연결은 여전히 인스펙터 UnityEvent
- 피해 층은 아직 ②

③에서 남은 일 · 조심할 것

1. **PatternContext 비대화 주의** — 벌써 7칸. ①의 `BossSkillContext`가 걸은 길과 같음. 보편(누가·대상·토큰)만 두고 나머지는 보스 참조를 통해 꺼내게
2. **남은 시그널 5개 이관** — 평타·Skill1~3은 아직 `CastAbility`. 피해를 주는 동작은 데미지 파이프라인과 함께 옮김
3. **옛 코드 삭제** — `UseSkill` · `UseAttack` · `_skills` · 플래그 2개 · `CastAbility`
4. **패턴 공유가 필요해지면** — 스텝을 SO + `SerializeReference` 데이터로, 런타임 상태는 러너로 옮김
5. **소환 목록 정리** — 먼저 죽은 소환몹이 목록에 남아, 풀 재사용 시 다른 몹을 같이 죽일 위험

한 줄 정리

①→② = **'무엇을'을 데이터로** (문자열 → 타입 있는 참조) · ②→③ = **'언제'를 표로, 섞인 목록을 층으로** · 다음 = **'어떻게'** (데미지 파이프라인).